

Lab Assignment 6:

LEMMA – Installation, Test and Modeling

The aim of this lab assignment is to set up the LEMMA modeling environment on your computer, to gain experience with the various modeling languages and to model your applications architecture. The implementation of LEMMA uses the Eclipse IDE. The LEMMA folder in the ILIAS course already contains prepared video to set up the modeling environments for Windows, Linux and MacOS.

Task 1 – Installation and Testing

1. Carry out the installation of LEMMA according to the installation video.
2. Import the LEMMA examples into your modeling environment and check that no errors occurred during the import.
3. Test the syntax highlighting and auto-completion.

Task 2 - Domain Data Modeling

These models should represent the domain concepts of your architecture. Also consider the concepts of Domain-driven Design (DDD) from which the design of your architecture is based. In other words, apply LEMMA's syntax for bounded contexts, entities, domain services etc. to the reconstructed domain concepts, provided this is consistent with your architecture. The domain models should also contain any technology information. As you have not learned how to add information from technology models to domain models using LEMMA in this semester, add a comment to the elements concerned. The following Java code is given as an example:

```
@Entity
@Table(name = "addresses") public class Address {
    @GeneratedValue @Id
    private Long id;

    ...
}
```

Figure 1: Domain Data Modeling - 1

Now model the JPA annotations shown as comments in the associated LEMMA domain model:

```
// Entity
// Table(name = „addresses“)
structure Address
  // GeneratedValue
  // Id
  long id,
  ...
}
```

Figure 2: Domain Data Modeling - 2

Task 2- Microservice Modeling

These models should represent the microservices of your architecture. Create one model each for a set of related microservices. For example, if you have programmed two microservices for a bounded context, model these microservices in a LEMMA service model that also refers or should refer to the associated domain model of the bounded context. You have learned in the lecture how to add technology information to LEMMA service models. In ILIAS you will find prepared LEMMA technology models for the lab assignment. Please note that in addition to the technology models, the ZIP archive also contains the PDF file “LEMMA- Technologycatalog.pdf”. This lists the technology elements (aspects, deployment technologies, protocols, etc.) for each of the prepared technology models. If none of the elements from the prepared technology models fit a technology you are using, you can use the generic technology model `Generic.technology` as an exception. You can find examples of this technology model in section 2 in the above-mentioned file “LEMMA-Technologycatalog.pdf”.

Task 3 - Deployment Modeling

These models should represent the containers and infrastructure nodes (API gateways, databases, service discoveries, etc.) of your architecture. If you have not created containers for your services, model a container for each microservice using the technology model `Docker.technology`, which you have downloaded from ILIAS as described above. The container can then be modeled in the same way as the LEMMA operation model.

```

@technology(Docker)
@technology(...) // Some protocol technology
container ServiceContainer
  deployment technology Docker::_deployment.Docker
  deploys Services::org.example.Service { aspects {
    Docker::_aspects.Dockerfile(
      '
      CMD java ${JAVA_OPTS} -jar order-service.jar
      COPY $$$PATH_TO_YOUR_SERVICE_JAR_NAME$$$jar .
      '
    );
  }
  default values {
    basic endpoints { ...
  }
}
}

```

Figure 3: Operation Modeling - 2

Replace the string “PATH_TO_YOUR_SERVICE_JAR_NAME” in the last line in the Dockerfile aspect with the actual file name of the JAR archive of your microservice. Furthermore, model the basic endpoints of your service analogous to the sample solutions. If none of the predefined protocols from the PDF file “LEMMA- Technologycatalog.pdf” are suitable for the endpoints, use one of the generic* protocols described in section 2 of the PDF file.

Evaluation criteria

- Syntactic correctness of the LEMMA models.
- Consistency of the LEMMA models with the source code of the backend microservices of your architecture
 - Consistent modeling of domain information, including DDD information.
 - Consistent modeling of microservices, their interfaces, operations, endpoints, etc.
 - Consistent modeling of containers and infrastructure nodes.

Hand-In

Commit the LEMMA code to the same repository as your Java architecture. Please store the models in a folder called "lemma" at the top file system level of your repository.